

Practical AI Tooling Tips For The Too Busy Developer

Audience

- Looking for ideas how to best leverage AI
- Been too busy shipping value to pay attention
- This'll be a 100-200 level talk
- More than just "AI in your IDE"
- This an all day workshop, but we have 60 mins

Poll

- Who's using AI tools? Every day?
- Copilot?
- Claude Code?
- Cursor?
- Codex?
- Windsurf?
- Something else?

Agenda

- Am I still going to have a job?
- Define all the AI Buzzwords like Context, Instructions, Skills, and more
- Talk about tools like Copilot and Claude Code
- Show some demos
- Practical Tips about where AI fits and where it doesn't

Goals

- Give you ideas you can start using today
- Where AI tooling is going in the future

Who am I?

- Director of Engineering at [Lean TECHniques](#)
- [Microsoft MVP](#)
- [Dometrain Author](#)
- Redgate Community Ambassador
- Co-organizer of [Iowa .NET User Group](#)



**Am I going to be
out of a job?**

History Lesson (2000s – Present)

- What set teams apart was their commitment to modern engineering practices and modern work practices – proven by [DORA](#)
- Fast feedback
- Working in small batches
- Learning Culture
- Automated Tests
- CI/CD
- Cloud
- Database Automation
- Monitoring and Observability
- And more...

**Which ones are still
relevant in an
AI-dominated world?**

History Lesson (2000s – Present)

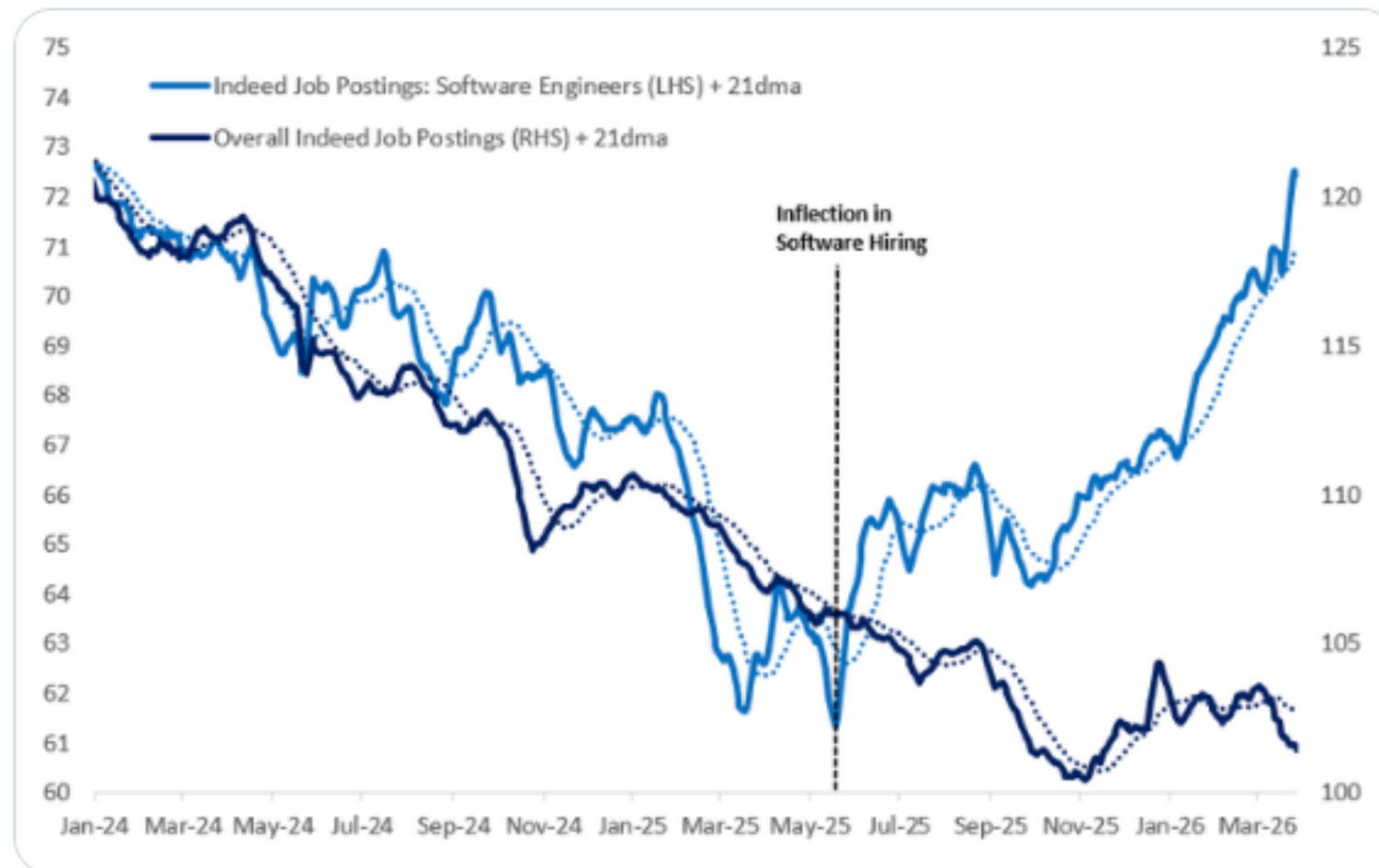
- What set teams apart was their commitment to modern engineering practices and modern work practices
- Fast feedback 
- Working in small batches 
- Learning Culture 
- Automated Tests 
- CI/CD 
- Cloud 
- Database Automation 
- Monitoring and Observability 
- And more...

**The Fundamentals
Remain The Same**

Jevon's Paradox

- Economic Principle of increasing the the efficiency of a resource actually leads to more use, not less
- More fuel efficient engines, means cheaper travel, which encourages people to drive, which means more fuel consumption
- Faster Internet speeds has led to more internet usage not less (ie streaming)
- If history repeats itself - becoming more efficient with creating software will actually result in MORE software, not less
- Building faster changes the Build vs Buy equation
- We all still have jobs!

Narrative violation and great insight from the latest Citadel Securities banger by Frank Flight: "We illustrated back in February that demand for software engineers, the most AI exposed occupation was accelerating higher, which we argued violates the displacement narrative. Indeed the acceleration in software job postings has continued, now up 18% from the inflection point in May last year."



“SaaS is Dead.”

- Satya Nadella, Microsoft CEO



**These tools are the
worst they will ever be
today.**

**Alright let's talk
AI Tools**

Claude writes 99% of
my code,
I still review all of it

AI is an Amplifier

- Good practices – move faster at high quality
- Bad practices – move faster at lower quality
- How do you verify your app works? Compile, Tests, Lint, Format. Are you doing those things already?
- Do you have judgement to know when AI went off the rails?
- Your PR output should 10x when using AI correctly
- Do your PRs sit for days without reviewing already?
- Do you rubberstamp PRs?
- You still own the code – not AI

Large Language Models

- Engine trained on massive amounts of text to learn patterns of meaning
- Literally just predicts one token at a time based on probabilities
- Often people just say Models
- Best Models is a misnomer – it's tradeoffs
- Accuracy vs Cost vs Speed
- Claude Opus 4.7 is the “best” model for one-shotting code in my experience, but it's the most expensive and often slow
- Haiku responds very quickly, but isn't as deep

Context

- The information an LLM is allowed to consider for a request
- LLMs have a limited Context Window
- What makes up Context:
 - Conversation History
 - Code in your repo
 - User Prompt
 - Instructions
- Clearing context is key – when one task is done, start a new chat/session
- Compacting can also lead to hallucinations

AI Dev Tools

- Unifies LLMs with codebases to provide the most context to get the best results
- GitHub Copilot (most ubiquitous)
- Claude Code (best AI dev tool)
 - Preview Feb 2025, GA May 2025
- Cursor (best autocomplete)
- Codex – haven't used but hear good things
- Lots of others
- Best == my opinions and experiences
- Autocomplete is nice... but there's so much more...

Modes

- Ask
 - Answers a question. ChatGPT
- Edit
 - “Just change this”
 - Limited Context, but you know exactly what needs to change
- Plan
 - Formulates your Prompt to feed to Agent/Edit mode
 - Asks you questions to clarify intent
 - You should be using Plan mode for any large changes
- Agent / Agentic / Auto
 - “Implement this feature...” “Fix the failing tests”

Instructions

- Set of Markdown files to define your team's/organization's rules
- “Don't use Arrange, Act, Assert comments in tests”
- Tip: anything the agent does wrong? Add it to your Instructions
 - This is key

Instructions

- From

Formatting

- Apply code-formatting style defined in `.editorconfig`.
- Prefer file-scoped namespace declarations and single-line using directives.
- Insert a newline before the opening curly brace of any code block (e.g., after `if`, `for`, `while`, `foreach`, `using`, `try`, etc.).
- Ensure that the final return statement of a method is on its own line.
- Use pattern matching and switch expressions wherever possible.
- Use `nameof` instead of string literals when referring to member names.
- Place private class declarations at the bottom of the file.

Nullable Reference Types

- Declare variables non-nullable, and check for `null` at entry points.
- Always use `is null` or `is not null` instead of `== null` or `!= null`.
- Trust the C# null annotations and don't add null checks when the type system says a value cannot be null.

Building

Always run restore first to set up the local SDK. Run `./restore.sh` (Linux/macOS) or `./restore.cmd` (Windows) first to install the local SDK. After restore, you can use standard `dotnet` commands, which will automatically use the local SDK when available due to the paths configuration in `global.json`.

Prerequisites

1. **Restore First:** Always run `./restore.sh` (Linux/macOS) or `./restore.cmd` (Windows) to set up the local .NET SDK (~30 seconds)

Build Commands

- **Full Build:** `./build.sh` (Linux/macOS) or `./build.cmd` (Windows) - defaults to restore + build (~3-5 minutes)
- **Build Only:** `./build.sh --build` (assumes restore already done)

INSTRUCTIONS

INSTRUCTIONS EVERYWHERE

Instructions Everywhere

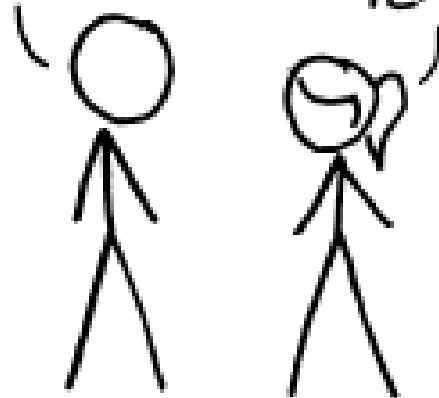
- Every tool now respects AGENTS.md in the root
- .cursorrules, .cursor/rules
- CLAUDE.md
- GEMINI.md
- .github/instructions/copilot-instructions.md

HOW STANDARDS PROLIFERATE:

(SEE: A/C CHARGERS, CHARACTER ENCODINGS, INSTANT MESSAGING, ETC)

SITUATION:
THERE ARE
14 COMPETING
STANDARDS.

14?! RIDICULOUS!
WE NEED TO DEVELOP
ONE UNIVERSAL STANDARD
THAT COVERS EVERYONE'S
USE CASES.



SOON:

SITUATION:
THERE ARE
15 COMPETING
STANDARDS.

Skills

- Rather than loading all those Instructions into context every time, what if we just did it selectively as needed?
- Saves context, saves tokens, saves money
- .github/skills/<skill name>/SKILL.md
- .claude/skills/<skill name>/SKILL.md
 - Recognized by Copilot too
- .agent/skills/<skill name>/SKILL.md
- Just became a thing in October
- <https://github.com/anthropics/skills>

Skills

```
skills > frontend-design > ⚡ SKILL.md
```

```
1  ---
2  name: frontend-design
3  description: Create distinctive, production-grade frontend interfaces with high design quality. Use this skill when the
   user asks to build web components, pages, artifacts, posters, or applications (examples include websites, landing
   pages, dashboards, React components, HTML/CSS layouts, or when styling/beautifying any web UI). Generates creative,
   polished code and UI design that avoids generic AI aesthetics.
4  license: Complete terms in LICENSE.txt
5  ---
6
7  This skill guides creation of distinctive, production-grade frontend interfaces that avoid generic "AI slop"
   aesthetics. Implement real working code with exceptional attention to aesthetic details and creative choices.
8
9  The user provides frontend requirements: a component, page, application, or interface to build. They may include
   context about the purpose, audience, or technical constraints.
10
11 ## Design Thinking
12
13 Before coding, understand the context and commit to a BOLD aesthetic direction:
14 - Purpose: What problem does this interface solve? Who uses it?
```

Copilot Coding Agent (and friends)

- Agents can run in the browser, not just on your machine
- Assign it a task or issue, watch progress (if you want), review the PR
- Create specialized agents for certain tasks – just more Instructions files
- Practical tip: Feature Toggle Remover agent
- This isn't coming to Azure DevOps

Demo

Code Reviews

- Copilot Code Reviews / Claude Code Reviews – you should be doing this
- ~80% of the comments are valid in my experience
- This isn't coming to Azure DevOps either

Demo

Workflow

- Plan extensively so you can “1 shot”
- Most Plans only take a few minutes to create
- Use the grill-me skill to enumerate all possibilities

My Workflow

- I'm pretty vanilla, but this workflow allows me to do anything in <60 minutes, most <15 minutes
- Usually I'm running 2-4 Claudes at once

My Workflow

- 🧑 Plan with Grill Me (ideally with a Jira story as a starting point)
- 🧑 Approve Plan
- 🤖 Creates checklist
- 🤖 Auto Mode to loop over checklist and write all the code
- 🤖 Loops over local verify script until it passes
- 🧑 Review code
- 🤖 Submit PR
- 🤖 Review PR comments with `/review-pr-comments`
- 🧑 Approve which PR comments we'll address
- 🤖 Auto Mode to fix PR comments
- 🧑 Review fixes and Merge

My Workflow

- AI writes 99% of my code
- I still review every single line
- I only use an IDE for diffs mostly

Demo

Where's it going

- More apps for managing like conductor.build – GitHub coming out with one too
- The bottleneck is moving from writing code to reviewing code
- The best AI-infused devs I know today spend 80-90% of their day reviewing code, not writing it
- Context window expanding

Top 10 Tips

- Give AI a way to validate itself - compile, tests, format, lint, acceptance criteria, code quality, etc
 - The Fundamentals don't change
- Start with Plan Mode by default - graduate out of it when you feel comfortable
- Auto Mode via `claude --enable-auto-mode`
- Skills
- CLIs over MCP servers - GitHub, Atlassian, Azure, etc
- Be aggressive clearing context
- Tighten your feedback loop (local and CI), then when you're done, tighten it again
- Use /insights - but don't be a sheep and accept everything
- Use /review and /security-review
- Keep your [AGENTS.md](#) lean - cross cutting things, how to verify your app, not your code structure, delegate to Skills
- Keep an 🧐 on AI
- Just Try Things

How do I stay up to date?

- Follow people on Twitter / LinkedIn
- @bcherny – Boris Cherny who created Claude Code
- @leeerob – Lee Robinson, Cursor
- @code – VS Code
- @claudeai – Claude
- @claude devs – Claude Devs
- @housecor – Cory House, consultant

Impact

- “We are moving at least 2x as fast with at least 2x the quality”
- “This makes me 10x faster”



1w • Edited •

After decades in the workforce, I didn't expect to feel this kind of momentum again.

Lately, I've been deep in the world of AI-assisted development—using tools like Claude for coding and code reviews, GitHub Actions automation, and building smarter workflows. Not just experimenting, but actually reshaping how I think, build, and deliver.

What's surprised me most isn't just the speed—it's the shift in mindset.

I'm learning how to:

- Write better prompts that produce cleaner, more maintainable code
- Use AI to pressure-test ideas before they become problems
- Build efficient workflows while the model “thinks,” instead of waiting on it
- Blend automation with human judgment so quality doesn't drop

There's a real skill in knowing when to lean on AI—and when not to.

This doesn't feel like a trend. It feels like a fundamental shift in how great work gets done. And if you're not actively learning how to work with these tools, you're going to fall behind people who are.

Still early. Still learning. But the energy is real—and I haven't felt this kind of excitement about the future of work in a long time.

Takeaways

- Ideas on where AI is and where it's going
- Ideas you can start using literally today
- Now's the time to jump in, because the productivity is real
- "Change has never been this fast but will never be this slow again."

Resources

- This slide deck at <https://scottsauber.com>

Questions?

ssauber@leantechniques.com

Add me on LinkedIn:



@scottsauer



@scottsauer.com

Slides at scottsauer.com

Thanks!

Add me on LinkedIn:



@scottsauer



@scottsauer.com

Slides at scottsauer.com